

Local Bindings

CMSC 323: Design and implementation of
programming languages

This lecture note is adapted from content created by Robby Findler (Northwestern University) and Vincent St-Amour (Northwestern University) and Shriram Krishnamurthi in Programming Languages: Application and Interpretation.

Formal Grammars

A formal grammar **specifies** which strings are valid and which aren't in our language

Formally, a *language* (not necessarily a programming language) is a collection of strings.

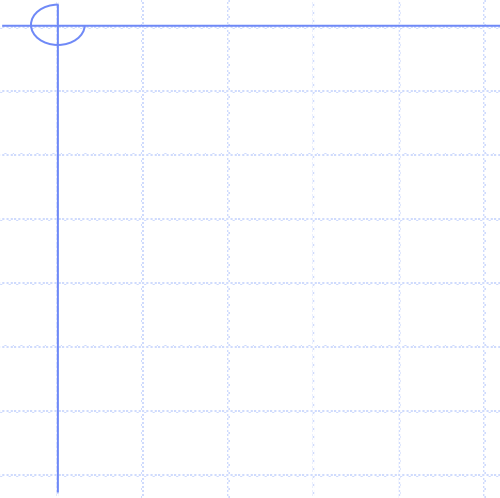
E.g., the language consisting of of all valid email addresses

For a programming language, we need to specify which programs (strings) are valid programs in the language.

Formal Grammars

Productions: Are the rules that specify the strings that may be produced

Derivations: are the strings we create from the grammar, using the productions



I am having a good day!

Day I am good having!

Context-Free Grammars

```
<expr> ::= <num>  
        | {+ <expr> <expr>}
```

The above expression is written in Backus-Naur Form (BNF). It is a convenient notation for the description of a language. We read it as define ($:=$) `expr` to be either a number or (|) the surface syntax consisting of an open brace ({), a plus sign (+), an `expr` and another `expr` and a close brace (}).

Aside: Context-Sensitive Grammars

In a Context-Free Grammar (CFG), a rule always looks like $A \rightarrow \gamma$. You can replace A no matter what is next to it.

In a **Context-Sensitive Grammar**, every production rule must match the following format:

$$\alpha A \beta \rightarrow \alpha \gamma \beta$$

Where:

- A is a single **non-terminal symbol** (the variable you want to rewrite).
- γ is a non-empty string of terminals and/or non-terminals ($\gamma \neq \epsilon$).
- α and β are strings of terminals and/or non-terminals that form the **context**.

This means A can only transform into γ **if and only if** it is directly preceded by α and directly followed by β . The context (α and β) remains entirely unchanged by the replacement.

Backus-Naur Form (BNF)

Each rule has a head and a body

`<expr> ::= <num>`

`| {+ <expr> <expr>}`

Head

body

Backus-Naur Form (BNF)

BNFs have **terminals** and **non-terminals**

Non-terminals are placeholders like **expr** and **num**. They stand for many more possibilities. They will be further expanded. By convention, they are placed between angle brackets.

Terminals (literals) are concrete syntax like **{, }** and **+** in our example

Backus-Naur Form

You may have multiple rules for the same name

$\langle \text{expr} \rangle ::= \langle \text{num} \rangle$

$\langle \text{expr} \rangle ::= \{ + \langle \text{expr} \rangle \langle \text{expr} \rangle \}$

Backus-Naur Form

Rules may refer to themselves

$\langle \text{expr} \rangle ::= \langle \text{num} \rangle$

$\langle \text{expr} \rangle ::= \{ + \langle \text{expr} \rangle \langle \text{expr} \rangle \}$

Backus-Naur Form

Use pipes to specify alternatives rules for a head

```
<expr> ::= <num>  
         | {+ <expr> <expr>}
```

Backus-Naur Form

Use star for repetition – 0 or more times

$\langle \text{fundef} \rangle ::= \{ \text{lam } \{ \langle \text{var} \rangle^* \} \langle \text{expr} \rangle \}$

Use + for 1 or more times

$\langle \text{fundef} \rangle ::= \{ \text{deffun } \{ \langle \text{var} \rangle^+ \} \langle \text{expr} \rangle \}$

Backus Naur form is used to describe the surface syntax

Example

{+ 1 2}

Program

AST

Result

Parser

Interpreter

Object
Language

Meta Language

Meta Language

Language used

Example Languages

DrRacket

Scheme

Scheme

Java

C, C++ and other languages

C, C++ and other languages

Python

C

C

Assembly

Machine Code

Machine Code

C

Assembly Language

Assembly Language

Backus-Naur Form

Given the following backus Naur form, is $\{1 + 1\}$ a valid expressions in our language:

```
<expr> ::= <num>
          | {+ <expr> <expr>}
          | {- <expr> <expr>}
          | {with {<var> <expr>} <expr>}
          | <var>
```

Backus-Naur Form

Given the following backus Naur form, is $\{+ \ 1 \ 1\}$ a valid expressions in our language:

```
<expr> ::= <num>
          | {+ <expr> <expr>}
          | {- <expr> <expr>}
          | {with {<var> <expr>} <expr>}
          | <var>
```

Backus-Naur Form

Given the following backus Naur form, is {with x 2 x} a valid expressions in our language:

```
<expr> ::= <num>
          | {+ <expr> <expr>}
          | {- <expr> <expr>}
          | {with {<var> <expr>} <expr>}
          | <var>
```

Backus-Naur Form

Given the following backus Naur form, is `{with {x} 2 x} a` valid expressions in our language:

```
<expr> ::= <num>
          | {+ <expr> <expr>}
          | {- <expr> <expr>}
          | {with {<var> <expr>} <expr>}
          | <var>
```

Backus-Naur Form

Given the following backus Naur form, is `{with {x 2} x}` a valid expressions in our language:

```
<expr> ::= <num>
          | {+ <expr> <expr>}
          | {- <expr> <expr>}
          | {with {<var> <expr>} <expr>}
          | <var>
```

Backus-Naur Form

Given the following backus Naur form, is

`{+ 2 {with {x 5} x}}`

a valid expressions in our language:

```
<expr> ::= <num>
          | {+ <expr> <expr>}
          | {- <expr> <expr>}
          | {with {<var> <expr>} <expr>}
          | <var>
```

Backus-Naur Form

Given the following backus Naur form, is

ab

a valid expressions in our language:

```
<expr> ::= <num>
          | {+ <expr> <expr>}
          | {- <expr> <expr>}
          | {with {<var> <expr>} <expr>}
          | <var>
```

Backus-Naur Form

Given the following backus Naur form, is

150

a valid expressions in our language:

```
<expr> ::= <num>
          | {+ <expr> <expr>}
          | {- <expr> <expr>}
          | {with {<var> <expr>} <expr>}
          | <var>
```

Evaluating Local Binding

Binding means associating a name with a value. E.g., a formal parameter with actual parameter or an identifier with a value.

Local is another way to refer to the scope of the name. Local binding means that the bound name, parameter or identifier is limited to some region of the program.

Static Scoping in Our Language

A variable's binding is determined by its position in the source program, and not by the order of program execution.

We determine the scope by following the structure of the AST.

Static scoping is a determining characteristic of SMOL

A Syntax for Our Local Binding

```
<expr> ::= <num>
         | {+ <expr> <expr>}
         | {with {<var> <expr>} <expr>}
         | <var>
```

- We create a local binding by using the `with` symbol. The first `<expr>` is evaluated and associated with the `<var>`. The second `<expr>` is evaluated and returned
 - The scope of the `<var>` is in the second `<expr>`

Static Scoping in Our Language

What do the following programs evaluate to?

a. `{with {x 1}
 {with {y 2}
 {with {x 3}
 {+ x y}}}}`

b. `{with {x 1}
 {+ x {with {x 2}
 x}}}}`

c. `{with {x 1}
 {+ {with {x 2} x}
 x}}`

d. `x`

Dynamic Scoping

The decision to let control flow determine binding is called
dynamic scoping.

Dynamic Scoping

- ◆ Dynamic scoping is an unequivocally bad design decision to make.
- ◆ With dynamic scoping, you cannot be sure about the binding structure of your program
- ◆ Your evaluator cannot be sure, either
- ◆ Programming tools cannot be sure

Dynamic Scoping

- ◆ There is a discussion in the text of the issues with dynamic scope. I suggest you take a look.

Evaluating Local Binding

What is the result of these programs in Python and Java assuming *i* has not been declared anywhere else in the program other than as displayed below?

```
for i in range(5):  
    pass  
  
print(i)
```

```
for (int i = 0; i < 5; i++);  
{  
    System.out.print(i);  
}
```

Examples

```
x = eval(input("enter a number: "))
```

```
if x > 0:
```

```
    y = 2
```

```
print(y)
```

```
int x = 1;
```

```
if (x > 0)
```

```
{
```

```
    int y = 6;
```

```
}
```

```
System.out.println(y);
```

For the Python code, `y` has a value sometimes and other times it does not

For the Java code, `y` does not have a value outside its scope. The Java program

snippet will always error since `y` is accessed outside its scope.

Including Local Binding In Our AST

Expanding our AST

WAE means “with arithmetic expression”

```
(define-type WAE
  [num (n : Number)]
  [add (left : WAE)
       (right : WAE)]
  [sub (left : WAE)
       (right : WAE)]
  [with (id : Symbol)
        (value : WAE)
        (body : WAE)]
  [var (s : Symbol)])
```

Including Local Binding in Our AST

Expanding our AST

```
(define-type WAE
  [num (n : Number)]
  [add (left : WAE)
        (right : WAE)]
  [sub (left : WAE)
        (right : WAE)]
  [with (id : Symbol)
         (value : WAE)
         (body : WAE)]
  [var (s : Symbol)])
```

with receives 3 parameters in its constructor:

id: A symbol for which a binding will occur

value: A WAE (an expression in our language)

that will be associated with **id**

body: An expression that is the scope of **id**

Including Local Binding in Our AST

Expanding our AST

```
(define-type WAE
  [num (n : Number)]
  [add (left : WAE)
       (right : WAE)]
  [sub (left : WAE)
       (right : WAE)]
  [with (id : Symbol)
        (value : WAE)
        (body : WAE)]
  [var (s : Symbol)])
```

Example 'with' expression:

```
(with 'x (num 10) (num 12))
```

In the above example, a binding occurs. 'x' is bound to a number 10

The body of the 'with' simply returns a number 12.

There is no reference to 'x' in the body of the
with

Including Local Binding in Our AST

Expanding our AST

```
(define-type WAE
  [num (n : Number)]
  [add (left : WAE)
       (right : WAE)]
  [sub (left : WAE)
       (right : WAE)]
  [with (id : Symbol)
        (value : WAE)
        (body : WAE)]
  [var (s : Symbol)])
```

- `var` is the constructor for a variable. i.e. We use `var` to create a variable in our language
 - The `var` constructor receives a symbol

Including Local Binding in Our AST

Expanding our AST

```
(define-type WAE
[num (n : Number)]
[add (left : WAE)
      (right : WAE)]
[sub (left : WAE)
      (right : WAE)]
[with (id : Symbol)
      (value : WAE)
      (body : WAE)]
[var (s : Symbol)])
```

Example **var** expression:

```
(var 'x)
```

A **free variable** has been created. Free variables is a reference to (use of) a variable that has no binding.

Including Local Binding in Our AST

Expanding our AST

```
(define-type WAE
[num (n : Number)]
[add (left : WAE)
      (right : WAE)]
[sub (left : WAE)
      (right : WAE)]
[with (id : Symbol)
      (value : WAE)
      (body : WAE)]
[var (s : Symbol)])
```

Example `var` expression:

```
(with 'x (num 10) (var 'x))
```

A binding for `x` is created, associating `x` with the value 10.

We can refer to `x` in the body of the `with`

Exercise

```
(define-type WAE
[num (n : Number)]
[add (left : WAE)
     (right : WAE)]
[sub (left : WAE)
     (right : WAE)]
[with (id : Symbol)
      (value : WAE)
      (body : WAE)]
[var (s : Symbol)])
```

Given the PLAIT type to the left,
express the surface syntax in the
next slide as ASTs.

Exercise

a. {with {x 1}
 {with {y 2}
 {with {x 3}
 {+ x y}}}}

b. {with {x 1}
 {+ x {with {x 2}
 x}}}}

c. {with {x 1}
 {+ {with {x 2} x}
 x}}

d. x

Free and Bound Identifiers

An identifier for the argument of a function or the name of a local identifier is a **binding occurrence**.

```
(define (f x) (+ x z))
```

```
(local [(define a 3)  
        (define c 4)]  
  (+ a b))
```

Free and Bound Identifiers

A use of a function argument or a local identifier is a **bound occurrence**.

```
(define (f x) (+ x z))
```

```
(local [(define a 3)
        (define c 4)]
  (+ a b))
```

Free and Bound Identifiers

A use of an identifier that is not a function argument or local identifier is a **free identifier**

```
(define (f x y) (+ x (+ y z)))
```

```
(local [(define a 3)
        (define c 4)]
  (+ a (+ b c)))
```

Shadowing

Shadowing happens when a binding occurrence of an identifier is already bound (i.e. there was a prior binding occurrence).

```
(define (f x y)
  (local [(define x 3)]
    (+ x y)))
```

Shadowing

Shadowing happens when a binding occurrence of an identifier occurs in a context where that identifier is already bound (i.e, there was a prior binding occurrence)

```
(define (f x y)
  (local [(define x 3)]
    (+ z y)))
```

This is **still** an example of shadowing; two binding **occurrences** of **x** even though **x** is not used.

Shadowing

Shadowing happens when a binding occurrence of an identifier occurs in a context where that identifier is already bound (i.e, there was a prior binding occurrence)

```
(+ (local [(define x 3)] x)
   (local [(define x 4)] x))
```

This is not an example of shadowing; two binding **occurrences** have non-overlapping (nested) scopes.

Exercise: Now In Our Language (WAE)

For each identifier in the expressions below, label them as either a binding occurrence, a bound occurrence, a free variable, and/or a shadowed variable.

a. `{with {x 1}
 {with {y 2}
 {with {x 3}
 {+ x y}}}}`

b. `{with {x 1}
 {+ x {with {x 2}
 x}}}}`

c. `{with {x 1}
 {+ {with {x 2} x}
 x}}`

d. `x`

Exercise: Now In Our Language (WAE)

For each identifier in the expressions below, label them as either a binding occurrence, a bound occurrence, a free variable, or a shadowed variable.

e. `{with {x 1}
 {+ x {with {x 2}
 y}}}}`